PRIORITY-INVERSION

# 우선순위 역전·상속·천장

연관 기출 · 126-4-2

## 한눈에 이해

급한 작업이 덜 급한 작업의 잠금을 기다리는데, 중간 순위 작업까지 끼어들어 지연이 길어지는 문제입니다.

## 정의와 핵심 원리

우선순위 역전은 높은 우선순위 작업이 낮은 우선순위 작업이 점유한 자원 때문에 막히는 현상입니다. 중간 우선순위 작업이 낮은 작업을 선점하면 높은 작업의 대기 시간이 간접적으로 늘어납니다.

• 우선순위가 T1 > T2 > T3이고 이진 세마포어 S = 1이라고 가정합니다. T3가 P(S)로 임계 구역에 들어간 뒤 T1이 같은 자원을 기다립니다. T3가 V(S)로 자원을 풀어야 T1이 실행할 수 있는데, 자원이 필요 없는 T2가 T3를 선점하면 T1의 대기도 길어집니다.

• 우선순위 상속에서는 T1이 기다리는 동안 잠금 소유자 T3의 유효 우선순위를 T1 수준까지 높입니다. 잠금이 여러 개 얽히면 상속된 우선순위가 대기 사슬을 따라 전파될 수 있습니다. 잠금을 풀 때는 남은 대기자와 원래 우선순위를 반영합니다.

• 우선순위 천장 방식은 자원을 이용할 작업 중 최고 우선순위에 기반한 천장과 진입 규칙을 사용합니다. 획득 즉시 올리는 방식 등 변형이 있으므로 천장의 정의·상승 시점·진입 조건을 함께 씁니다. 짧은 임계 구역과 일관된 잠금 순서도 필요합니다.

## 구조·비교·판단

| 구분 | 핵심 원리 | 적용·한계 |
| --- | --- | --- |
| 상속 | 높은 우선순위 작업이 기다릴 때 소유자의 우선순위를 높임 | 동적 대응, 상속이 이어지는 대기 사슬 관리 |
| 천장 | 자원별 최고 우선순위 기반 제한 | 사전 이용 관계 분석 필요 |
| 단순 우선순위 | 실행 가능한 최고 작업 선점 | 잠금 의존성 자체 해결 못함 |

## 적용 예제

### 세 작업의 P/V 순서

강사 생성 학습 예제입니다. 공식 기준답안이 아닙니다.

**조건:** 단일 CPU를 사용하는 T1·T2·T3의 우선순위는 각각 높음·중간·낮음입니다.

1. T3:P(S) → 임계 구역 시작.

2. T1:P(S) → 대기.

3. T2가 실행되어 T3를 지연시키면 역전이 확대됩니다.

4. 우선순위 상속이 적용되면 T3의 우선순위를 T1 수준으로 올립니다. T3가 임계 구역의 일을 마치고 V(S)로 자원을 반환하면 T1이 다시 실행됩니다.

5. T3의 유효 우선순위는 잠금과 대기 사슬 상태에 맞게 복원합니다.

**결과·해석:** 고우선순위 작업의 간접 차단을 제한합니다. 모든 종류의 지연과 교착이 자동 제거되지는 않습니다.

**전제·주의:** Linux rt_mutex의 실제 상속 구현은 확인했습니다. 천장 프로토콜의 모든 변형은 원표준 추가 확인 대상입니다.

## 답안 골격과 혼동 방지

실시간 응답 문제 → T1/T2/T3 시간축 → P/V 흐름 → 상속·천장 비교 → 블로킹 상한

- 낮은 작업이 자원을 가진 사실 자체를 모두 비정상이라 하지 않습니다.
- 상속을 영구 우선순위 변경이라 쓰지 않습니다.

## 내용을 가리고 회상

1. T2는 공유 자원을 쓰지 않는데도 왜 T1의 실행을 늦춥니까?

2. 상속을 언제 복원합니까?

3. 천장에 사전 자원 이용 정보가 필요한 이유는 무엇입니까?

## 적용 기준과 근거

명시한 출처 판본의 원리를 설명하며 제품별 구현 차이는 별도 확인합니다.

**내용 검토 상태: 부분 확인**. 출처 확인은 공식 정답이나 학습 성취를 뜻하지 않습니다.

출처별 상세 열람 범위는 위키의 출처 기록에서 확인합니다.


- RT-mutex implementation design - Steven Rostedt / Linux Kernel; 2006 저작권 표기; 확인 2026-11-05.
- Lock types and their rules - Linux Kernel documentation; 상시 문서; 확인 2026-11-05.